ASSIGNMENT 13.5

T.Harivarshith
2303A51351
Batch 20

Task Description #1 (Refactoring – Removing Global Variables) • Task:
Use AI to eliminate unnecessary global variables from the code.
• Instructions:
o Identify global variables used across functions. o Refactor the code to pass values using function parameters. o Improve modularity and testability.
• Sample Legacy Code:
rate = 0.1 def calculate_interest(amount): return amount * rate print(calculate_interest(1000)) • Expected Output: o Refactored version passing rate as a parameter or using a configuration structure. **PROMPT:**
#rate = 0.1
#def calculate_interest(amount):
#return amount * rate
#print(calculate_interest(1000)
#eliminate unnecessary global variables from the above code and make the function more flexible by allowing the caller to specify the interest rate as a parameter.

CODE:

```
def calculate_interest(amount, rate=0.1):
    """Calculate interest based on the given amount and rate."""
    return amount * rate
#example usage:
interest = calculate_interest(1000, rate=0.05)
print(interest)
```

OUTPUT:

```
50.0
```

OBSERVATION:
The function `calculate_interest` is now more flexible as it allows the caller to specify the interest rate as a parameter. If no rate is provided, it defaults to 0.1 (10%). This eliminates the need for a global variable and makes the function more reusable in different contexts where different interest rates may be applicable.

Task Description #2 : (Refactoring Deeply Nested Conditionals) • Task:
Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.
• Focus Areas: o
Readability o Logical simplification o Maintainability Legacy
Code: score = 78 if score >= 90: print("Excellent")
else: if score >= 75:
print("Very Good")
else: if score >= 60:
print("Good") else:
print("Needs Improvement") Expected Outcome:
o Flattened logic using guard clauses or a mapping-based approach.

PROMPT:
#score = 78 #if
score >= 90:
#print("Excellent")
#else:
#if score >= 75:
#print("Very Good") #else:
#if score >= 60:
#print("Good") #else:
#print("Needs Improvement")
#efactor deeply nested if–elif–else logic into a cleaner structure. Readability Logical simplification Maintainability Flattened logic using guard clauses or a mapping-based approach. **CODE:**

```
def evaluate_score(score):
    """Evaluate the score and return a corresponding message."""
    if score >= 90:
        return "Excellent"
    elif score >= 75:
        return "Very Good"
    elif score >= 60:
        return "Good"
    else:
        return "Needs Improvement"
#example usage:
result = evaluate_score(78)
print(result)
```

OUTPUT:

```
Very Good
```

OBSERVATION:

The function `evaluate_score` has been refactored to use a cleaner structure with `elif` statements, which improves readability and maintainability. The logic is straightforward and easy to follow, making it clear how the score is evaluated against the different thresholds.

Task 3 (Refactoring Repeated File Handling Code)

• Task: Use AI to refactor repeated file open/read/close logic.

• Focus Areas: o DRY principle o Context managers o Function reuse Legacy Code:

f = open("data1.txt")
print(f.read())
f.close() f =
open("data2.txt")
print(f.read())
f.close()

Expected Outcome:

o Reusable function using with open() and parameters.

PROMPT:

#f = open("data1.txt")
#print(f.read())
#f.close()
#f = open("data2.txt")
#print(f.read())
#f.close()

refactor the repeated file open/read/close logic . focus areas: dry principle,context managers,function reuse

CODE:

```
def read_file(filename):
    try:
        with open(filename, encoding="utf-8") as f:
            return f.read()
    except FileNotFoundError:
        return f"Error: {filename} not found"
# Example usage:
data1 = read_file("data1.txt")
print(data1)
data2 = read_file("data2.txt")
print(data2)
```

OUTPUT:

```
Error: data1.txt not found
Error: data2.txt not found
```

OBSERVATION

The `read_file` function encapsulates the logic for opening, reading, and closing a file using a context manager (`with` statement). This approach adheres to the DRY principle by eliminating repeated code and ensures that files are properly closed even if an error occurs. Additionally, it includes error handling for cases where the file may not be found, improving the robustness of the code.

Task 4 (Optimizing Search Logic)

- Task: Refactor inefficient linear

searches using appropriate data structures.

- Focus Areas: o Time complexity o Data

structure choice Legacy Code:

users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ") found = False for u in

users: if u == name: found = True print("Access Granted"

if found else "Access Denied") Expected Outcome:

o Use of sets or dictionaries with complexity justification.

PROMPT:

#users = ["admin", "guest", "editor", "viewer"]

#name = input("Enter username: ") #found =

False

#for u in users: #if

u == name:

#found = True

#print("Access Granted" if found else "Access Denied")

#refacor inefficient linear searches using appropriate data structures focus mainly on time complexity and data structure choice

CODE:

```
def check_access(username, users):
    """Check if the username exists in the users list."""
    return "Access Granted" if username in users else "Access Denied"
#example usage:
users = ["admin", "guest", "editor", "viewer"]
username = input("Enter username: ")
result = check_access(username, users)
print(result)
```

OUTPUT:

```
Enter username: admin
Access Granted
```

OBSERVATION:

The function `check_access` has been refactored to use the `in` keyword, which is more efficient for checking membership in a list. However, if the list of users is large, it would be more efficient to use a set instead of a list, as sets have O(1) average time complexity for membership checks compared to O(n) for lists.

Task 5 (Refactoring Procedural Code into OOP Design) • Task:
Use AI to refactor procedural code into a class-based design.
• Focus Areas: o Object-
Oriented principles
o Encapsulation Legacy Code: salary =
50000 tax = salary * 0.2 net = salary - tax
print(net)
Expected Outcome:
o A class like EmployeeSalaryCalculator
with methods and attributes.

PROMPT:
#salary = 50000
#tax = salary * 0.2
#net = salary - tax
#print(net)

#refactor the above code into class-based design . focus areas: object-oriented design, encapsulation . explected outcome; a class like employeesalarycalculator with methods and attributes

CODE:

```python
class EmployeeSalaryCalculator:
    """A class to calculate net salary after tax deductions."""

    def __init__(self, salary, tax_rate=0.2):
        self.salary = salary
        self.tax_rate = tax_rate

    def calculate_tax(self):
        """Calculate the tax based on the salary and tax rate."""
        return self.salary * self.tax_rate

    def calculate_net_salary(self):
        """Calculate the net salary after deducting tax."""
        tax = self.calculate_tax()
        return self.salary - tax
# Example usage:
calculator = EmployeeSalaryCalculator(salary=50000)
net_salary = calculator.calculate_net_salary()
print(net_salary)
```

OUTPUT:

```
40000.0
```

OBSERVATION:

The `EmployeeSalaryCalculator` class encapsulates the logic for calculating tax and net salary. It has attributes for salary and tax rate, and methods to calculate tax and net salary. This design promotes object-oriented principles such as encapsulation and makes the code more organized and reusable.

Task 6 (Refactoring for Performance Optimization)

- Task: Use AI to refactor a performance-heavy loop handling large data.
- Focus Areas: o Algorithmic optimization

o Use of built-in functions Legacy Code:

total = 0 for i in range(1,
1000000): if i % 2 == 0:
total += i print(total)

Expected Outcome:

o Optimized logic using mathematical formulas or comprehensions,
with time comparison.

PROMPT:

#total = 0
#for i in range(1, 1000000):
#if i % 2 == 0:
#total += i
#print(total)
#refactor a performance heavy loop handling large data . focus area: Algorithmic optimization and use of built in functions .

CODE:

```
def sum_even_numbers(n):
    """Calculate the sum of even numbers from 1 to n."""
    return sum(i for i in range(1, n) if i % 2 == 0)
# Example usage:
result = sum_even_numbers(1000000)
print(result)
```

OUTPUT:

```
249999500000
[10, 20]
```

OBSERVATION:

The function `sum_even_numbers` uses a generator expression within the built-in `sum` function to calculate the sum of even numbers from 1 to n. This approach is more efficient than using a loop and

manually accumulating the total, as it leverages Python's optimized built-in functions and avoids the overhead of explicit loops.

Task 7 (Removing Hidden Side Effects)

• Task: Refactor code that modifies shared mutable state.

• Focus Areas: o Functional-style refactoring

o Predictability Legacy Code:

data = [] def

add_item(x):

data.append(x)

add_item(10)

add_item(20)

print(data)

Expected Outcome:

o Refactored function returning values

instead of mutating globals.

PROMPT:

#data = []

#def add_item(x):

#data.append(x)

#add_item(10)

#add_item(20)

#print(data)

#refactor the above code to modify shared mutable state . focus area: functional style refactoring , predictability

CODE:

```
def add_item(data, x):
    """Return a new list with the item added, without modifying the original list."""
    return data + [x]
# Example usage:
data = []
data = add_item(data, 10)
data = add_item(data, 20)
print(data)
```

OUTPUT:

```
[10, 20]
```

OBSERVATION:

The `add_item` function has been refactored to return a new list with the item added, rather than modifying the original list. This approach promotes immutability and functional programming principles, making the code more predictable and easier to debug, as it avoids side effects associated with mutable state.

Task 8 (Refactoring Complex Input Validation Logic)

- Task: Use AI to simplify and modularize complex validation rules.
- Focus Areas: o Readability o Testability Legacy

Code: password = input("Enter password: ") if len(password) >= 8: if any(c.isdigit() for c in password): if any(c.isupper() for c in password): print("Valid Password") else: print("Must contain uppercase") else: print("Must contain digit") else: print("Password too short") Expected Outcome:

o Separate validation functions with clear responsibility

PROMPT:

#password = input("Enter password: ") #if len(password) >= 8:
#if any(c.isdigit() for c in password):
#if any(c.isupper() for c in password):
#print("Valid Password") #else:
#print("Must contain uppercase") #else:
#print("Must contain digit") #else:
#print("Password too short")
#refactor the code to simplify and modularize complex validation rules . focus Area: readability , testability . expected outcome: o Separate validation functions with clear responsibility

CODE:

```python
def is_valid_password(password):
    """Check if the password meets the validation criteria."""
    if len(password) < 8:
        return "Password too short"
    if not any(c.isdigit() for c in password):
        return "Must contain digit"
    if not any(c.isupper() for c in password):
        return "Must contain uppercase"
    return "Valid Password"
# Example usage:
password = input("Enter password: ")
result = is_valid_password(password)
print(result)
```

OUTPUT:

```
Enter password: akk
Password too short
PS C:\Users\reddy\OneDrive\Desktop\AI ass>
```

OBSERVATION:

The `is_valid_password` function has been refactored to check each validation criterion separately, improving readability and maintainability. Each check is clear and concise, making it easier to understand the validation logic. This modular approach also allows for easier testing of each individual validation rule.